

# RIVALS: AI TRACKING & COMPUTER VISION MANUAL

Complete Technical Architecture: Biomechanical Mathematics, Multi-Person Spatial Tracking, AR Holographic Overlays, and Zero-Calibration Engine

**DOCUMENT SAFETY & DECOUPLING NOTICE**

This PDF is a 100% standalone technical reference manual. It is completely decoupled from your Flutter codebase. You can open it, study it, share it, or delete it at any time without having any effect on the app functionality.

|                       |                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------|
| Target System:        | RIVALS Mobile App (Flutter / Dart)                                                  |
| Core Computer Vision: | Google ML Kit BlazePose Neural Model (On-Device 3D Pose Inference)                  |
| Key Inventions:       | Dual Mode (Side / Front Floor), Multi-Person Spatial Matching, Floating AR Nametags |
| Signal Architecture:  | Exponential Moving Average (EMA) + Schmitt-Trigger Hysteresis FSM                   |
| Author & Version:     | Antigravity AI Engineering • Production Edition 2.5                                 |

# 1. Biomechanical Mathematics & Formulas

A major flaw in basic fitness apps is applying 2D trigonometry to every camera angle. RIVALS solves this with two distinct mathematical models tailored to the exact physical viewpoint of the camera.

## Mode A: Side Profile View (3-Point Joint Vector Trigonometry)

When the phone is placed to the side (e.g. on a stand or bottle), the arm moves in the 2D sagittal plane. We compute the exact internal angle formed at the elbow vertex B between Shoulder A and Wrist C using the vector dot product formula:

```
// Vector Angle Formula:
// Vector BA: Shoulder(A) -> Elbow(B) => BA = (Ax - Bx, Ay - By)
// Vector BC: Wrist(C) -> Elbow(B) => BC = (Cx - Bx, Cy - By)

double calculateAngle(PoseLandmark a, PoseLandmark b, PoseLandmark c) {
    double v1x = a.x - b.x; double v1y = a.y - b.y;
    double v2x = c.x - b.x; double v2y = c.y - b.y;

    double dotProduct = (v1x * v2x) + (v1y * v2y);
    double mag1 = sqrt((v1x * v1x) + (v1y * v1y));
    double mag2 = sqrt((v2x * v2x) + (v2y * v2y));

    if (mag1 * mag2 == 0) return 180.0;
    double cosine = (dotProduct / (mag1 * mag2)).clamp(-1.0, 1.0);
    return acos(cosine) * (180.0 / pi); // Outputs 0° to 180°
}
```

- **Top Lockout:** Elbow Angle  $\geq 155^\circ$  (arms extended).
- **Bottom Inflection (Depth):** Elbow Angle  $\leq 95^\circ$  (chest close to floor).

## Mode B: Front Floor View (Normalized Vertical Compression Ratio)

When the phone is flat on the floor in front of the user, the arm moves in the Z-axis (depth). 2D angles break because the arm foreshortens. RIVALS calculates the **Normalized Vertical Compression Ratio**, measuring chest-to-wrist travel divided by torso length:

```
// Distance-Invariant Compression Formula:
double computeFrontDepthRatio(double shoulderY, double wristY, double hipY) {
    double verticalDelta = (wristY - shoulderY).abs(); // Chest-to-floor distance
    double torsoLength = (hipY - shoulderY).abs(); // Scale reference

    if (torsoLength < 10) return 1.0;
    return (verticalDelta / torsoLength).clamp(0.0, 1.2);
}
// Top Position: Ratio ~ 0.75 - 0.90 | Bottom Position: Ratio <= 0.35
```

## Spine & Plank Alignment Formula (Form Quality Scoring)

To check if the user is maintaining a straight body without hip sag or piking, the hip Y-coordinate is compared against the linear midpoint between shoulders and ankles:

```
// Sag / Pike Deviation Formula:  
double totalSpan = (ankleY - shoulderY).abs();  
double expectedHipY = shoulderY + (totalSpan * 0.5);  
double deviation = (hipY - expectedHipY).abs() / totalSpan;  
  
bool isSpineStraight = deviation <= 0.25; // Sag tolerance limit
```

## 2. Signal Processing & Schmitt-Trigger FSM

Raw camera frames produce sensor noise and micro-jitter. RIVALS uses a two-tier signal pipeline: **Exponential Moving Average (EMA)** followed by a **4-Stage Schmitt-Trigger Finite State Machine**.

### A. Temporal Exponential Moving Average (EMA)

Filters high-frequency frame jitter without adding perceptible lag:

```
// EMA Formula (alpha = 0.35):
double applyEma(double currentSample, double? previousFilteredValue, double alpha = 0.35) {
    if (previousFilteredValue == null) return currentSample;
    return (alpha * currentSample) + ((1.0 - alpha) * previousFilteredValue);
}
```

### B. The 4-Stage Schmitt-Trigger State Machine

A single threshold line causes rapid duplicate counts when someone pauses at the bottom. By separating entrance and exit boundaries by a 60° hysteresis buffer, double-counting is mathematically prevented:

| State          | Trigger Condition                          | Action Output                                               |
|----------------|--------------------------------------------|-------------------------------------------------------------|
| 1. TOP (Ready) | Angle $\geq 155^\circ$ / Ratio $\geq 0.75$ | Ready for descent. Reset per-rep metrics.                   |
| 2. DESCENDING  | Angle $< 140^\circ$ / Ratio $< 0.65$       | Eccentric phase confirmed. Monitoring depth.                |
| 3. BOTTOM      | Angle $\leq 95^\circ$ / Ratio $\leq 0.35$  | Full depth achieved! Flag valid depth.                      |
| 4. ASCENDING   | Angle $> 115^\circ$ / Ratio $> 0.45$       | Concentric phase. Pushing up.                               |
| 5. COMPLETED   | Angle returns $\geq 155^\circ$             | <b>REP COUNTED!</b> Calculate Form Score & trigger Haptics. |

## 3. Multi-Person Spatial Tracking & Dynamic Auto-Joining

RIVALS supports simultaneous tracking of multiple athletes. The core engine is built in `lib/services/multi\_pose\_tracker.dart` using **Spatial Centroid Matching**.

### A. Spatial Centroid & Left-to-Right Sorting

Each detected skeleton has its horizontal center computed across the Nose, Shoulders, and Hips. Poses are sorted from Left (0.0) to Right (1.0) on the screen:

```
double computePoseCentroidX(Pose pose, double imageWidth) {
    final keyLandmarks = [pose.landmarks[PoseLandmarkType.nose], pose.landmarks[PoseLandmarkType.leftShoulder], pose.landmarks[PoseLandmarkType.rightShoulder]];
    double sumX = 0; int count = 0;
    for (final lm in keyLandmarks) {
        if (lm != null && lm.likelihood > 0.3) { sumX += lm.x; count++; }
    }
    return (sumX / count) / imageWidth; // Normalized 0.0 (Left) to 1.0 (Right)
}
```

### B. Dynamic Auto-Joining & Seamless Transition

- **Solo State:** Only 1 person is detected. Tracked as Player 1 with a clean solo HUD.
- **Friend Steps In:** When ML Kit outputs a 2nd pose, the tracker creates a Player 2 slot. Player 1's score is uninterrupted while Player 2 starts counting independently.
- **Absence Grace Period (TTL):** If an athlete steps out for water or rests on knees, their slot is held in memory for 45 seconds so they never lose their count upon returning.

## 4. AR Canvas Overlay & Camera Matrix Alignment

To make skeleton lines and holographic AR nametags stick directly to real human body parts without offset, `lib/painters/posePainter.dart` uses an exact **BoxFit.cover** **matrix transformation** matching front camera mirroring and rotated sensor buffers.

```
// Exact BoxFit.cover Coordinate Translation:
Offset translate(double x, double y, Size size, Size imageSize, bool isFrontCamera) {
  double scale = max(size.width / imageSize.width, size.height / imageSize.height);
  double scaledWidth = imageSize.width * scale;
  double scaledHeight = imageSize.height * scale;
  double offsetX = (size.width - scaledWidth) / 2;
  double offsetY = (size.height - scaledHeight) / 2;

  // Mirror X coordinate for natural selfie camera view
  double effectiveX = isFrontCamera ? (imageSize.width - x) : x;

  return Offset(offsetX + (effectiveX * scale), offsetY + (y * scale));
}
```

### Floating AR Holographic Nametag Rendering

In multiplayer mode, floating nametags track above each athlete's head using canvas text painters and frosted rounded pill rectangles:

```
// Floating Nametag above Head:
final headPos = translate(nose.x, nose.y, size);
final tagCenter = Offset(headPos.dx, headPos.dy - 38);

// Draw frosted background pill + neon border + text (Name • Reps)
canvas.drawRRect(pillRect, bgPaint);
canvas.drawRRect(pillRect, borderPaint);
textPainter.paint(canvas, textOffset);
```

## 5. Complete Modular Codebase Architecture

The application is structured into clear, decoupled layers ensuring zero spaghetti code and easy future expansion:

| File Path                              | Layer           | Single Responsibility                                                  |
|----------------------------------------|-----------------|------------------------------------------------------------------------|
| `lib/main.dart`                        | Entry Point     | Initializes bindings, portrait orientation lock, and dark gold theme.  |
| `lib/app.dart`                         | Navigation      | Maintains tab state via IndexedStack and custom center camera trigger. |
| `lib/services/pose_service.dart`       | Biomechanics    | Trigonometric angles, compression ratio, EMA smoothing, and FSM.       |
| `lib/services/multi_pose_tracker.dart` | Multi-Person    | Spatial X-centroid sorting, slot allocation, absence TTL memory.       |
| `lib/painters/posePainter.dart`        | AR Presentation | Draws multi-player neon glowing skeletons and floating AR nametags.    |
| `lib/screens/camera_screen.dart`       | Feature UI      | Glassmorphic HUD, Torch/Flip controls, and workout summary modal.      |

**MODIFICATION & DELETION SAFETY**

Every component in this codebase is strictly modular. The PoseService is independent of Flutter UI; the MultiPoseTracker is independent of the camera hardware. You can modify any math formula, adjust colors, or delete this PDF at any time with zero impact on the rest of the application.